

Esercizi su MiniCaml

Esercizio 1. Estendere il linguaggio MiniCaml e l'implementazione dell'interprete con il nuovo costrutto **bifun** che consente di definire funzioni anonime non ricorsive con due parametri formali (non Curried: al momento dell'applicazione dovranno essere passati esattamente due argomenti a tali funzioni). Ad esempio, in una sintassi concreta simile a quella di OCaml, il costrutto **bifun** potrebbe essere usato come segue:

```
bifun x y -> x+y
```

Una funzione così definita è a tutti gli effetti un valore, quindi può essere usata anche in un **let** (già presente in MiniCaml). Ad esempio (sempre in sintassi OCaml):

```
let somma = bifun x y -> x + y
in ....
```

L'esistenza di questo nuovo tipo di funzione richiede anche di definire un nuovo costrutto di applicazione per funzioni a due parametri (**Apply** non va bene per queste funzioni, perchè prevede un unico argomento). In questo modo si potrebbe completare l'esempio in questo modo (sempre in sintassi OCaml):

```
let somma = bifun x y -> x + y
in somma 3 2
```

Esercizio 2. Estendere il linguaggio MiniCaml e l'implementazione dell'interprete con il nuovo tipo di dato "lista di interi" con le seguenti operazioni (già in sintassi astratta):

- **Empty** descrive una lista di interi vuota
- **Cons(e1,e2)**, dove **e1** descrive un intero e **e2** una lista, produce la lista ottenuta aggiungendo l'intero descritto da **e1** in testa alla lista descritta da **e2**
- **Length(e)**, dove **e** descrive una lista, calcola la lunghezza di tale lista
- **SumAll(e)**, dove **e** descrive una lista, calcola la somma degli elementi di tale lista

Ad esempio (in sintassi astratta):

<code>Cons(EInt(1),Cons(EInt(2),Empty))</code>	<code>(* viene valutato in un valore che descrive la lista [1;2] *)</code>
<code>Length(Cons(EInt(1),Cons(EInt(2),Empty)))</code>	<code>(* viene valutato in Int(2), poiché la lista contiene 2 elementi *)</code>
<code>SumAll(Cons(EInt(1),Cons(EInt(2),Empty)))</code>	<code>(* viene valutato in Int(3), poiché la la somma degli elementi è 3 *)</code>